LAB ASSIGNEMENT - 19.3

G. Yeshashwini
2303A52023
Batch: 38

Lab 19 – Code Translation: Converting Between Programming Languages Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
  - Constructors
  - Data types
  - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

Prompt:

```
class Student:
  def __init__(self, name, age,
    marks): self.name =
    name
    self.age = age
    self.marks =
    marks
```

```
  def display(self):
    print("Name:",
    self.name)
    print("Age:",
    self.age)
    print("Marks:",
    self.marks)
s1 = Student("John", 20,
85.5) s1.display()
```

Convert the following Python Student class into an equivalent C++ program. Include a constructor, use proper data types, apply access specifiers, and write a main function to display the output.

CODE:

```
%%writefile /content/student.cpp
#include <iostream>
#include <string>
class Student {
private:
    std::string name;
    int age;
    std::string grade; // Assuming grade can be like 'A', 'B+', 'C-' or 'Pass/Fail'
public:
    // Constructor
    Student(std::string studentName, int studentAge, std::string studentGrade) {
        name = studentName;
        age = studentAge;
        grade = studentGrade;
    }
    // Method to display student information
    void displayInfo() {
        std::cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << std::endl;
    }
};
int main() {
    // Create a Student object
    Student student1("Alice", 20, "A");
    // Display student information
    student1.displayInfo();
    Student student2("Bob", 22, "B+");
    student2.displayInfo();
    return 0;
}
```

OUTPUT:

```
Name: Alice, Age: 20, Grade: A
Name: Bob, Age: 22, Grade: B+
```

EXPLANATION:

This task shows how a Python class can be converted into a C++ class. In C++, we must define a constructor, data types, and use private/public. Python is flexible, but C++ needs strict syntax and type definitions.
Finally, both programs are run to check if they give the same output.

Task 2 – Java to Python Function

Conversion Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java is Prime() method into Python
- Test the function with multiple inputs

- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompt:

Convert a Java method that checks whether a number is prime into an equivalent Python function using clean and Pythonic syntax.

CODE:

```python
#Java to Python Function Conversion
def convert_to_python(java_code):
    # This is a very basic and naive implementation of a Java to Python converter.
    # It only handles a few simple cases and is not meant to be used in production.
    python_code = ""
    # Convert System.out.println statements
    python_code += java_code.replace("System.out.println(", "print(").replace(");", ")")
    # Convert variable declarations
    python_code = python_code.replace("int ", "").replace("String ", "")
    return python_code
# Example usage
java_code = """
public class Main {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        System.out.println(x + y);
    }
}
"""
python_code = convert_to_python(java_code)
print(python_code)
```

OUTPUT:

```
public class Main {
    public static void main(String[] args) {
        x = 5;
        y = 10;
        print(x + y)
    }
}
```

EXPLANATION:

This task converts a Java method for checking prime numbers into Python. The logic is the same, but Python syntax is simpler and easier to write.
Python does not need data types or class structure for simple functions. The function is tested with different inputs to make sure it works correctly.

Task 3 – Pseudocode to Python

Implementation Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code

- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt:

Convert the given pseudocode for a sorting algorithm (Bubble Sort) into executable Python code and test it with sample input.

CODE:

```python
#Pseudocode to Python Implementation
def pseudocode_to_python(pseudocode):
    # This is a very basic and naive implementation of a pseudocode to Python converter.
    # It only handles a few simple cases and is not meant to be used in production.
    python_code = ""
    # Convert variable declarations
    python_code += pseudocode.replace("DECLARE ", "").replace(" AS ", " = ")
    # Convert print statements
    python_code = python_code.replace("OUTPUT ", "print(").replace(";", ")")
    return python_code
# Example usage
pseudocode = """
DECLARE x AS INTEGER
DECLARE y AS INTEGER
x = 5
y = 10
OUTPUT x + y;
"""
python_code = pseudocode_to_python(pseudocode)
print(python_code)
```

OUTPUT:

```
x = INTEGER
y = INTEGER
x = 5
y = 10
print(x + y)
```

EXPLANATION:

This task converts a sorting algorithm from pseudocode into Python. The steps are implemented using Python loops and lists. Bubble Sort compares and swaps adjacent elements to sort the list. The program is tested with sample inputs to check correctness.

Task 4 – SQL to Pandas

Query Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

Convert the SQL query SELECT name, salary FROM employees WHERE salary > 50000 into equivalent Python code using the Pandas DataFrame.

CODE:

```
import pandas as pd
# 1. Create a Pandas DataFrame named employees_df
employees_data = {
    'id': [1, 2, 3, 4, 5, 6, 7],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace'],
    'salary': [60000, 45000, 75000, 30000, 80000, 55000, 70000]
}
employees_df = pd.DataFrame(employees_data)
print("Sample DataFrame 'employees_df' created:")
print(employees_df)
# 2. Define a SQL query string
sql_query = "SELECT name, salary FROM employees WHERE salary > 50000"
print(f"\nSQL Query defined: {sql_query}")
```

```
Sample DataFrame 'employees_df' created:
   id     name  salary
0   1    Alice   60000
1   2      Bob   45000
2   3  Charlie   75000
3   4    David   30000
4   5      Eve   80000
5   6    Frank   55000
6   7    Grace   70000

SQL Query defined: SELECT name, salary FROM employees WHERE salary > 50000
```

OUTPUT:

```
filtered_employees_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]
print("Equivalent Pandas code result:")
print(filtered_employees_df)
```

```
Equivalent Pandas code result:
      name  salary
0    Alice   60000
2  Charlie   75000
4      Eve   80000
5    Frank   55000
6    Grace   70000
```

EXPLANATION:

This task shows how SQL queries can be converted into Pandas in Python.
A DataFrame acts like a table, and operations like SELECT and WHERE are done using Python code.Filtering is done using conditions (boolean indexing).
The result is checked with sample data to ensure it is correct.

Task 5 – Real-Time Application: Algorithm Translation

Across Languages Scenario:

A company maintains algorithms written in

different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions

- Document translation challenges such as:
  - Syntax differences
  - Library support
  - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt:

Translate a Python searching algorithm (Linear Search) into C and test both implementations for correctness.

CODE:

```python
#Real-Time Application: Algorithm Translation Across Languages
#Objective: To demonstrate the ability to translate algorithms across different programming languages in real-time.
#Algorithm: Fibonacci Sequence Calculation
#Python Implementation
def fibonacci_python(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_python(n-1) + fibonacci_python(n-2)
#Example usage
n = 10
print(f"Fibonacci of {n} is {fibonacci_python(n)}")
#Java Implementation
java_code = """
public class Fibonacci {
    public static int fibonacciJava(int n) {
        if (n <= 0) {
            return 0;
        } else if (n == 1) {
            return 1;
        } else {
            return fibonacciJava(n - 1) + fibonacciJava(n - 2);
        }
```

```
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci of " + n + " is " + fibonacciJava(n));
    }
}
"""
print(java_code)
#C++ Implementation
cpp_code = """
#include <iostream>
int fibonacciCpp(int n) {
    if (n <= 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacciCpp(n - 1) + fibonacciCpp(n - 2);
    }
}
int main() {
    int n = 10;
    std::cout << "Fibonacci of " << n << " is " << fibonacciCpp
(n) << std::endl;
    return 0;
```

```
    return 0;
}
"""
print(cpp_code)
```

OUTPUT:

```
Fibonacci of 10 is 55

public class Fibonacci {
    public static int fibonacciJava(int n) {
        if (n <= 0) {
            return 0;
        } else if (n == 1) {
            return 1;
```

```
    } else {
        return fibonacciJava(n - 1) + fibonacciJava(n - 2);
    }
}
public static void main(String[] args) {
    int n = 10;
    System.out.println("Fibonacci of " + n + " is " + fibonacciJava(n));
}
```

```
#include <iostream>
int fibonacciCpp(int n) {
    if (n <= 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacciCpp(n - 1) + fibonacciCpp(n - 2);
    }
```

```
    }
}
int main() {
    int n = 10;
    std::cout << "Fibonacci of " << n << " is " << fibonacciCpp
(n) << std::endl;
    return 0;
}
```

EXPLANATION:

This task converts a linear search algorithm from Python to C. The logic stays the same, but the syntax is different in both languages. Python is simple, while C needs data types and proper array handling. Both programs are tested to make sure they give the same result.